

# ASSIGNMENT 13

## Natural Language Processing, NLP Libraries, Word Embeddings, RNN, LSTM, GRU, Seq2Seq and Attention

---

### QUESTION 1

**What is Natural Language Processing? Explain any four real-world applications of NLP.**

**Answer:**

**Natural Language Processing (NLP)** is a branch of Artificial Intelligence and Machine Learning that enables computers to understand, process, analyze, and generate human language.

Human language can be in the form of:

- Text
- Speech
- Documents
- Emails
- Messages
- Social media posts

NLP combines techniques from **Computer Science, Linguistics, Machine Learning, and Deep Learning** to process human language.

### **Four Real-World Applications of NLP**

#### **1. Chatbots and Virtual Assistants**

NLP is used in applications such as customer-service chatbots and virtual assistants to understand user questions and generate appropriate responses.

Examples:

- Customer support chatbots
- Voice assistants
- AI conversational systems

## **2. Sentiment Analysis**

Sentiment analysis determines whether a text expresses a positive, negative, or neutral opinion.

Example:

"I really enjoyed this movie."

The system may classify it as:

Positive

## **3. Machine Translation**

NLP is used to automatically translate text from one language to another.

Example:

English → Hindi

English → French

English → Spanish

## **4. Spam Detection**

NLP can analyze email or message content and classify messages as:

Spam

or

Not Spam

## **Other Applications**

NLP is also used in:

- Text summarization
- Question answering
- Speech recognition
- Search engines
- Grammar checking

- Named Entity Recognition
  - Information extraction
  - Recommendation systems
- 

## Why Can a Machine Not Directly Understand Raw Text?

A computer fundamentally processes numerical representations, while raw human language consists of words, sentences, symbols, and linguistic structures.

For example:

"I love Python."

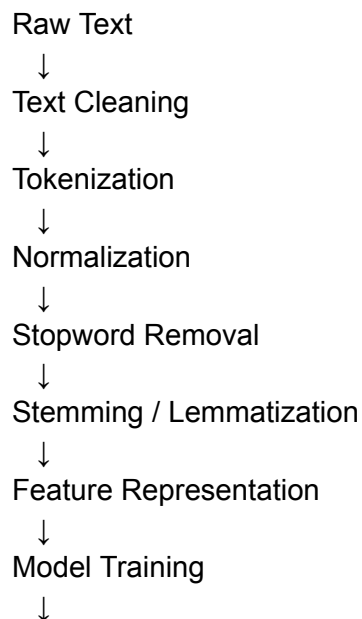
A computer cannot directly perform mathematical operations on the meaning of this sentence without converting the text into a suitable numerical representation.

Therefore, NLP converts text into structured or numerical representations that machine-learning algorithms can process.

---

## Basic NLP Pipeline

A basic NLP pipeline can contain the following stages:



## Explanation

1. **Text Collection** – Obtain text from documents, websites, messages, etc.
2. **Cleaning** – Remove unwanted characters, punctuation, or noise.
3. **Tokenization** – Divide text into words or tokens.
4. **Normalization** – Convert text into a consistent form.
5. **Stopword Removal** – Remove common words when appropriate.
6. **Stemming/Lemmatization** – Reduce words to base forms.
7. **Feature Representation** – Convert text into numerical features.
8. **Modeling** – Apply Machine Learning or Deep Learning.
9. **Evaluation** – Measure model performance.

## Easy Definition:

NLP enables computers to process and understand human language using computational and machine-learning techniques.

---

# QUESTION 2

**Explain the purpose of text preprocessing in NLP.  
Describe the following operations with suitable examples.**

Text preprocessing is the process of cleaning and transforming raw text into a consistent format before performing NLP analysis or training a Machine Learning model.

It helps reduce unnecessary noise and creates more useful input for downstream tasks.

---

## 1. Lowercasing

Lowercasing converts all text into lowercase.

### Example:

"Natural Language Processing"

becomes:

```
"natural language processing"
```

### **Python:**

```
text = "Natural Language Processing"
```

```
text = text.lower()
```

```
print(text)
```

### **Output:**

```
natural language processing
```

---

## **2. Removing Punctuation**

Punctuation marks such as:

```
! , . ? ; :
```

can be removed when they are not useful for the task.

### **Example:**

```
"Hello, World!"
```

becomes:

```
"Hello World"
```

### **Python:**

```
import string
```

```
text = "Hello, World!"
```

```
text = text.translate(  
    str.maketrans(  
        string.punctuation, ''  
    )  
)
```

```
        '''
        ,
        '''
        ,
        string.punctuation
    )
)

print(text)
```

---

### 3. Removing Numbers

Numbers can be removed when they are not relevant to the NLP task.

#### Example:

"I bought 2 books."

becomes:

"I bought books."

#### Python:

```
import re

text = "I bought 2 books."

text = re.sub(
    r"\d+",
    "",
    text
)

print(text)
```

---

### 4. Removing Stopwords

**Stopwords** are very common words that may provide limited information for certain NLP tasks.

Examples:

the  
is  
a  
an  
and  
of  
in  
to

However, stopword removal is **task-dependent**. For some tasks, removing words such as "not" can be harmful because it changes sentiment meaning.

### **Example:**

"The product is very good."

After removing some stopwords:

"product very good"

---

## **5. Handling Extra Whitespaces**

Multiple spaces can be converted into a single space.

### **Example:**

"Hello    World"

becomes:

"Hello World"

### **Python:**

```
text = "Hello    World"
```

```
text = " ".join(  
    text.split()  
)
```

```
print(text)
```

---

## 6. Tokenization

Tokenization divides text into smaller units called **tokens**.

### Example:

"I love Python"

becomes:

```
["I", "love", "Python"]
```

### Python:

```
from nltk.tokenize import word_tokenize
```

```
text = "I love Python."
```

```
tokens = word_tokenize(text)
```

```
print(tokens)
```

---

## 7. Stemming

Stemming reduces words to a root-like form using simple rules.

### Example:

playing → play

played → play

plays → play

However, stemming can sometimes produce words that are not valid dictionary words.

### Python:

```
from nltk.stem import PorterStemmer
```

```
stemmer = PorterStemmer()
```

```
words = [  
    "playing",  
    "played",  
    "plays"  
]
```

```
for word in words:  
    print(  
        word,  
        "→",  
        stemmer.stem(word)  
    )
```

---

## 8. Lemmatization

Lemmatization converts a word to its meaningful dictionary base form.

### Example:

better → good  
running → run  
studies → study

Lemmatization usually uses linguistic information such as word type/POS to obtain better results.

### Python:

```
from nltk.stem import WordNetLemmatizer
```

```
lemmatizer = WordNetLemmatizer()
```

```
print(  
    lemmatizer.lemmatize(  
        "studies",  
        pos="v"  
    )  
)
```

---

## 9. Unigrams, Bigrams and Trigrams

### Unigram

A unigram contains one word.

Example:

"I love NLP"

Unigrams:

I  
love  
NLP

### Bigram

A bigram contains two consecutive words.

I love  
love NLP

### Trigram

A trigram contains three consecutive words.

I love NLP

### Example:

For:

"I love natural language processing"

Unigrams:

I

love  
natural  
language  
processing

Bigrams:

I love  
love natural  
natural language  
language processing

Trigrams:

I love natural  
love natural language  
natural language processing

### Easy Definition:

Text preprocessing cleans and converts raw text into a suitable form for NLP algorithms.

---

## QUESTION 3

**Compare NLTK, spaCy, and Gensim based on their main purpose and common use cases.**

### Answer:

NLTK, spaCy, and Gensim are popular Python libraries used in NLP, but they have different strengths and purposes.

| Feature      | NLTK                                                   | spaCy               | Gensim                             |
|--------------|--------------------------------------------------------|---------------------|------------------------------------|
| Main Purpose | NLP education, text processing and linguistic analysis | Fast industrial NLP | Topic modeling and word embeddings |

|                   |                               |                    |                                      |
|-------------------|-------------------------------|--------------------|--------------------------------------|
| Tokenization      | Yes                           | Yes                | Available                            |
| POS Tagging       | Yes                           | Yes                | Limited compared with spaCy/NLTK     |
| Lemmatization     | Yes                           | Yes                | Not its main purpose                 |
| NER               | Yes                           | Yes                | Not its main focus                   |
| Word2Vec          | Can use related tools/modules | Not its main focus | Strong support                       |
| Topic Modeling    | Yes                           | Limited            | Strong                               |
| Ease for Learning | Very good                     | Good               | Good                                 |
| Production NLP    | Less optimized than spaCy     | Excellent          | Useful for embeddings/topic modeling |

## NLTK

**Natural Language Toolkit (NLTK)** is widely used for:

- Learning NLP concepts
- Tokenization
- Stemming
- Lemmatization
- POS tagging
- Corpus processing

## spaCy

spaCy is designed for efficient and production-oriented NLP.

It supports:

- Tokenization
- Lemmatization
- POS tagging
- Named Entity Recognition
- Dependency parsing

## Gensim

Gensim is particularly useful for:

- Word embeddings
  - Word2Vec
  - Doc2Vec
  - Topic modeling
  - Similarity analysis
- 

## What are Word Embeddings?

**Word embeddings** are dense numerical vector representations of words.

Words with similar meanings are generally represented by vectors that are closer together in the embedding space.

For example:

king  
queen  
man  
woman

can be represented as dense vectors.

---

## Word Embeddings vs Bag of Words vs TF-IDF

| Feature                | Bag of Words    | TF-IDF          | Word Embeddings                                             |
|------------------------|-----------------|-----------------|-------------------------------------------------------------|
| Representation         | Sparse          | Sparse          | Dense                                                       |
| Semantic relationships | Very limited    | Limited         | Better                                                      |
| Vector size            | Vocabulary size | Vocabulary size | Fixed embedding dimension                                   |
| Word order             | Usually ignored | Usually ignored | Basic Word2Vec is context-based but not full sentence order |

|                    |             |                      |               |
|--------------------|-------------|----------------------|---------------|
| Meaning similarity | Poor        | Limited              | Better        |
| Example            | Word counts | Weighted word counts | Dense vectors |

## Bag of Words

Represents a document using word occurrence/counts.

Example:

"I love NLP"

may become a vector based on vocabulary counts.

## TF-IDF

TF-IDF gives higher importance to words that are frequent in a document but relatively uncommon across the document collection.

## Word Embeddings

Word embeddings represent words using dense numerical vectors that can capture useful semantic and contextual relationships.

### Easy Definition:

**BoW and TF-IDF mainly represent word importance in documents, while word embeddings represent words as dense vectors that can capture semantic relationships.**

---

# QUESTION 4

**What is Sequential Data? Explain why an ordinary feed-forward neural network is not sufficient for many NLP tasks.**

**Answer:**

**Sequential data** is data in which the order of observations or elements is important.

Examples include:

- Sentences
- Text documents
- Speech
- Time series
- Stock prices
- Sensor readings

For example:

"I love Python"

has a different structure from:

"Python love I"

The order of words contributes to meaning.

---

## Why Feed-Forward Neural Networks Can Be Insufficient

A traditional feed-forward neural network generally processes its input without maintaining a hidden state representing previous sequence elements.

For many NLP tasks, understanding the current word requires information from previous or later words.

### Example:

"The bank is near the river."

versus:

"The bank approved my loan."

The word **bank** has different meanings depending on context.

A feed-forward network does not naturally maintain sequential context.

Therefore, models such as:

- RNN
- LSTM
- GRU
- Transformers

are commonly used for sequence-related tasks.

---

# Compare RNN, LSTM and GRU

## RNN – Recurrent Neural Network

RNN processes sequential data while maintaining a hidden state.

### Advantages:

- Simple architecture.
- Can process sequences.
- Maintains information from previous time steps.

### Limitation:

Standard RNNs can suffer from **vanishing and exploding gradient problems**, making long-term dependencies difficult to learn.

---

## LSTM – Long Short-Term Memory

LSTM is a specialized RNN architecture designed to better handle long-term dependencies.

It uses gates such as:

- Forget gate
- Input gate
- Output gate

### Advantages:

- Handles long-term dependencies better than basic RNN.
- Reduces the impact of vanishing gradients.

### Limitation:

- More parameters.
  - More computationally expensive than simple RNN.
- 

## GRU – Gated Recurrent Unit

GRU is another gated recurrent architecture.

It uses two main gates:

- Update gate
- Reset gate

### Advantages:

- Simpler than LSTM.
- Fewer parameters.
- Often faster to train than LSTM.

### Comparison:

| Feature                       | RNN              | LSTM             | GRU                     |
|-------------------------------|------------------|------------------|-------------------------|
| Basic recurrent architecture  | Yes              | Yes              | Yes                     |
| Gates                         | No               | 3 main gates     | 2 main gates            |
| Long-term dependency handling | Weak             | Stronger         | Strong                  |
| Parameters                    | Fewer            | More             | Usually fewer than LSTM |
| Complexity                    | Low              | High             | Medium                  |
| Training speed                | Generally faster | Generally slower | Often faster than LSTM  |

---

# Evaluation Metrics

## 1. Accuracy

Accuracy measures the proportion of total predictions that are correct.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

---

## 2. Precision

Precision measures how many predicted positive cases are actually positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

---

## 3. Recall

Recall measures how many actual positive cases were correctly identified.

$$\text{Recall} = \frac{TP}{TP + FN}$$

---

## 4. F1-Score

F1-score is the harmonic mean of precision and recall.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

---

## 5. Perplexity

**Perplexity** is commonly used to evaluate language models.

Lower perplexity generally indicates that the model assigns higher probability to the observed sequence.

Conceptually:

$$\text{Perplexity} = e^{-\frac{1}{N} \sum_{i=1}^N \log P(w_i)}$$

A lower value generally indicates better predictive performance for the evaluated language-modeling task.

### Easy Definition:

**RNN, LSTM and GRU process sequential information; LSTM and GRU use gates to handle long-term dependencies better than basic RNNs.**

---

## QUESTION 5

**What is a Sequence-to-Sequence model? Explain the roles of the encoder, decoder, hidden states, and context vector.**

### Answer:

A **Sequence-to-Sequence (Seq2Seq) model** is a neural-network architecture that converts one sequence into another sequence.

It is commonly used for:

- Machine translation
- Text summarization
- Question answering
- Conversational systems

### Example:

English Sentence



Encoder



Context / Representation



Decoder



Hindi Sentence

---

# 1. Encoder

The encoder processes the input sequence one element at a time and converts it into an internal representation.

Example:

"I love Python"

The encoder processes:

I → love → Python

---

# 2. Decoder

The decoder generates the output sequence step by step.

For example:

I love Python

may be translated into another language by generating output tokens sequentially.

---

# 3. Hidden States

Hidden states contain information learned from previous sequence elements.

They allow the recurrent network to maintain information while processing a sequence.

---

# 4. Context Vector

In a basic encoder-decoder model, the final encoder representation can act as a **context vector** summarizing the input sequence.

The decoder uses this representation to generate the output.

---

# Why Does Basic Seq2Seq Struggle with Long Sentences?

A basic encoder-decoder architecture may need to compress the entire input sequence into a fixed-size representation.

For a long sentence, important information from the beginning may be difficult to preserve.

This creates an information bottleneck.

---

## Attention Mechanism

The **attention mechanism** solves this problem by allowing the decoder to access different encoder hidden states when generating each output token.

Instead of relying only on one fixed context vector, the decoder can assign different weights to different input positions.

### Example:

Input:

"The cat is sitting on the mat."

↓

Decoder step 1

Focus → "The"

Decoder step 2

Focus → "cat"

Decoder step 3

Focus → "sitting"

...

The model dynamically focuses on relevant parts of the input.

### **Basic Attention Process:**

Encoder Hidden States



Attention Scores



Softmax



Attention Weights



Weighted Sum



Context Vector



Decoder

### **Advantages of Attention:**

- Handles long sequences better.
- Reduces the fixed-vector bottleneck.
- Allows the model to focus on relevant input words.
- Improves sequence-to-sequence performance.

### **Easy Definition:**

Seq2Seq converts one sequence into another, while attention allows the decoder to focus on the most relevant input positions at each step.

---

## **QUESTION 6**

### **Building a Reusable Text-Preprocessing Function**

#### **Given Text:**

" This PRODUCT is Amazing!!! I bought 2 items, and I really loved them. "

The program should:

- Convert text to lowercase.
- Remove punctuation.
- Remove numbers.
- Remove extra whitespaces.
- Remove English stopwords.
- Tokenize the cleaned text.
- Create a reusable preprocessing function.

## Google Colab Code:

```
# Install NLTK if required
```

```
!pip install -q nltk
```

```
import nltk
```

```
import re
```

```
import string
```

```
from nltk.corpus import stopwords
```

```
from nltk.tokenize import word_tokenize
```

```
# Download required resources
```

```
nltk.download("punkt")
```

```
nltk.download("punkt_tab")
```

```
nltk.download("stopwords")
```

```
def preprocess_text(text):
```

```
    # 1. Lowercase
```

```
    text = text.lower()
```

```
    # 2. Remove punctuation
```

```
    text = text.translate(
```

```
        str.maketrans(
```

```
            "",
```

```
            "",
```

```
            string.punctuation
```

```
        )
```

```
    )
```

```
    # 3. Remove numbers
```

```
    text = re.sub(
```

```
        r"\d+",
```

```
        "",
```

```
        text
```

```
    )
```

```

# 4. Remove extra whitespaces
text = " ".join(
    text.split()
)

# 5. Tokenization
tokens = word_tokenize(text)

# 6. Remove stopwords
stop_words = set(
    stopwords.words("english")
)

tokens = [
    word
    for word in tokens
    if word not in stop_words
]

return tokens

# Given text
text = (
    " This PRODUCT is Amazing!!! "
    "I bought 2 items, and I really loved them. "
)

# Apply function
cleaned_tokens = preprocess_text(text)

print("Cleaned Tokens:")
print(cleaned_tokens)

```

## Expected Output:

The output will contain cleaned tokens similar to:

```
['product', 'amazing', 'bought', 'items', 'really', 'loved']
```

The exact stopwords result depends on the NLTK English stopwords list/version.

## Explanation:

The function is reusable because the same `preprocess_text()` function can be applied to multiple text documents.

## Easy Definition:

A reusable preprocessing function performs the same text-cleaning steps consistently on different inputs.

---

# QUESTION 7

**Use NLTK to tokenize the sentence, apply Porter stemming, WordNet lemmatization, and generate unigrams, bigrams, and trigrams.**

## Given Sentence:

"The students were studying and running towards their classes."

## Google Colab Code:

```
!pip install -q nltk
```

```
import nltk
```

```
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
from nltk.util import ngrams
from nltk import pos_tag
```

```
# Download resources
nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("wordnet")
nltk.download("omw-1.4")
nltk.download("averaged_perceptron_tagger")
nltk.download("averaged_perceptron_tagger_eng")
```

```
# Sentence
sentence = (
    "The students were studying and running "
    "towards their classes."
)
```

```
# Tokenization
tokens = word_tokenize(
    sentence
)
```

```
print("Tokens:")
print(tokens)
```

```
# Porter Stemming
stemmer = PorterStemmer()
```

```
stemmed_words = [
    stemmer.stem(word)
    for word in tokens
]
```

```
print("\nPorter Stemming:")
print(stemmed_words)
```

```
# POS tagging
pos_tags = pos_tag(tokens)
```

```
print("\nPOS Tags:")
print(pos_tags)
```

```
# Convert POS tags to WordNet POS
from nltk.corpus import wordnet
```

```
def get_wordnet_pos(tag):
```

```
    if tag.startswith("J"):
        return wordnet.ADJ
```

```
    elif tag.startswith("V"):
        return wordnet.VERB
```

```
    elif tag.startswith("N"):
```

```

        return wordnet.NOUN

    elif tag.startswith("R"):
        return wordnet.ADV

    return wordnet.NOUN

# WordNet Lemmatization
lemmatizer = WordNetLemmatizer()

lemmatized_words = [
    lemmatizer.lemmatize(
        word,
        get_wordnet_pos(tag)
    )
    for word, tag in pos_tags
]

print("\nWordNet Lemmatization:")
print(lemmatized_words)

# Unigrams
unigrams = list(
    ngrams(tokens, 1)
)

print("\nUnigrams:")
print(unigrams)

# Bigrams
bigrams = list(
    ngrams(tokens, 2)
)

print("\nBigrams:")
print(bigrams)

# Trigrams
trigrams = list(
    ngrams(tokens, 3)
)

print("\nTrigrams:")

```

```
print(trigrams)
```

---

## Comparison of Stemming and Lemmatization

### Stemming

Stemming uses rule-based word reduction.

For example:

studying → studi

running → run

classes → class

A stem may not always be a valid dictionary word.

### Lemmatization

Lemmatization attempts to produce a meaningful dictionary base form.

Examples can include:

students → student

studying → study

running → run

classes → class

The exact result depends on the POS information supplied to the lemmatizer.

### Main Difference:

| Stemming                    | Lemmatization                          |
|-----------------------------|----------------------------------------|
| Faster                      | Usually more computationally involved  |
| Rule-based reduction        | Uses linguistic/dictionary information |
| May produce non-words       | Usually produces valid base forms      |
| Less linguistically precise | More linguistically meaningful         |

## Easy Definition:

Stemming cuts words to a root-like form, while lemmatization tries to find the correct dictionary base form.

---

# QUESTION 8

**Use the following documents to create Bag of Words and TF-IDF representations.**

## Documents:

Document 1:

"NLP is useful for text analysis"

Document 2:

"Text analysis is useful for business"

Document 3:

"NLP converts text into numerical features"

---

## Part A: Bag of Words

### Python Code:

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
documents = [  
    "NLP is useful for text analysis",  
    "Text analysis is useful for business",  
    "NLP converts text into numerical features"  
]
```

```
# Create Bag of Words  
bow_vectorizer = CountVectorizer()
```

```
X_bow = bow_vectorizer.fit_transform(  
    documents
```

```

)

# Vocabulary
print("Vocabulary:")
print(
    bow_vectorizer.vocabulary_
)

# Feature names
print("\nFeature Names:")
print(
    bow_vectorizer.get_feature_names_out()
)

# Feature matrix
print("\nBag of Words Matrix:")
print(
    X_bow.toarray()
)

```

### Explanation:

Bag of Words represents each document as a vector containing word counts.

---

## Part B: TF-IDF

### Python Code:

```

from sklearn.feature_extraction.text import TfidfVectorizer

# Create TF-IDF
tfidf_vectorizer = TfidfVectorizer()

X_tfidf = tfidf_vectorizer.fit_transform(
    documents
)

# Vocabulary
print("TF-IDF Vocabulary:")
print(

```

```
tfidf_vectorizer.get_feature_names_out()
)

# TF-IDF matrix
print("\nTF-IDF Matrix:")

print(
    X_tfidf.toarray()
)
```

---

## Identify the Word with Highest TF-IDF Score in Each Document

```
import numpy as np

feature_names = (
    tfidf_vectorizer
    .get_feature_names_out()
)

tfidf_matrix = X_tfidf.toarray()

for i, row in enumerate(
    tfidf_matrix
):

    max_index = np.argmax(row)

    word = feature_names[
        max_index
    ]

    score = row[
        max_index
    ]

    print(
        f'Document {i+1}: '
        f'{word} '
        f'(TF-IDF = {score:.4f})"
```

)

### Important Note:

The exact highest-scoring word should be taken from the program output because TF-IDF values depend on the vectorizer settings and corpus.

### Interpretation:

- **TF** measures how frequently a word occurs in a document.
- **IDF** reduces the importance of words that occur across many documents.
- **TF-IDF** gives higher weight to words that are important to a document relative to the collection.

### Formula:

$$\text{TFIDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

A common IDF form is:

$$\text{IDF}(t) = \log(N + 1 / \text{df}(t) + 1) + 1$$

where:

- $N$  = number of documents
- $\text{df}(t)$  = number of documents containing term  $t$

### Easy Definition:

**Bag of Words counts words, while TF-IDF gives greater importance to words that are relatively distinctive within a document collection.**

---

## QUESTION 9

### PART A: spaCy

#### Analyze the sentence:

"Microsoft opened a new AI research centre in Bengaluru in 2025."

Display:

- Tokens
- Lemmas
- Part-of-speech tags
- Named entities

## Step 1: Install spaCy

```
!pip install -q spacy
```

```
!python -m spacy download en_core_web_sm
```

## Step 2: Python Code

```
import spacy
```

```
# Load English model
```

```
nlp = spacy.load(  
    "en_core_web_sm"  
)
```

```
sentence = (  
    "Microsoft opened a new AI research "  
    "centre in Bengaluru in 2025."  
)
```

```
doc = nlp(sentence)
```

```
# Tokens
```

```
print("TOKENS:")  
for token in doc:  
    print(token.text)
```

```
# Lemmas
```

```
print("\nLEMMAS:")  
for token in doc:  
    print(  
        token.text,  
        "→",  
        token.lemma_  
    )
```

```
# POS Tags
```

```

print("\nPART-OF-SPEECH:")
for token in doc:
    print(
        token.text,
        "→",
        token.pos_
    )

# Named Entities
print("\nNAMED ENTITIES:")
for ent in doc.ents:
    print(
        ent.text,
        "→",
        ent.label_
    )

```

## Expected Entity Types

Depending on the spaCy model version, entities are expected to include entities such as:

```

Microsoft → ORG
Bengaluru → GPE
2025      → DATE

```

The exact output can depend on the installed spaCy model version.

## Explanation:

### Tokens

Tokens are individual text units.

### Lemmas

Lemmas are the base forms of words.

Example:

opened → open

### POS Tags

POS tagging identifies grammatical categories such as:

NOUN  
VERB  
ADJ  
PROPN  
ADP

## Named Entity Recognition

NER identifies real-world entities such as:

- Organizations
  - Locations
  - Dates
  - People
  - Products
- 

# PART B: Word2Vec

**Train a small Word2Vec model using Gensim and find words similar to "king".**

## Step 1: Install Gensim

```
!pip install -q gensim
```

## Step 2: Python Code

```
from gensim.models import Word2Vec
```

```
# Small training corpus
```

```
sentences = [  
    ["king", "queen", "royal", "palace"],  
    ["king", "man", "prince", "royal"],  
    ["queen", "woman", "princess", "royal"],  
    ["man", "woman", "person", "human"],  
    ["prince", "king", "royal", "male"],  
    ["princess", "queen", "royal", "female"],
```

```
["king", "queen", "prince", "princess"],  
["royal", "palace", "king", "queen"],  
["man", "boy", "king", "male"],  
["woman", "girl", "queen", "female"]  
]
```

```
# Train Word2Vec  
model = Word2Vec(  
    sentences=sentences,  
    vector_size=50,  
    window=3,  
    min_count=1,  
    workers=1,  
    epochs=200,  
    seed=42  
)
```

```
# Similar words  
print(  
    "Words similar to 'king':"  
)
```

```
print(  
    model.wv.most_similar(  
        "king",  
        topn=5  
    )  
)
```

## Explanation:

Word2Vec learns dense vector representations based on word context.

The `most_similar()` method finds words whose vectors are close to the vector of "king".

## Important Note:

Because this is a **very small artificial training corpus**, the exact similarity values and ranking are not expected to represent real-world language relationships. A large corpus would produce more meaningful embeddings.

## Easy Definition:

Word2Vec learns word vectors from surrounding context, allowing similar words to have similar vector representations.

---

## QUESTION 10

### PART A: LSTM Model

**Build a simple LSTM model in TensorFlow/Keras for binary sentiment classification.**

The model should contain:

- Text vectorization/tokenization
- Embedding layer
- LSTM layer
- Dense output layer

#### **Answer:**

We can use the IMDB movie-review dataset available through Keras.

The target is binary:

0 → Negative

1 → Positive

#### **Google Colab Code:**

```
import tensorflow as tf
```

```
from tensorflow.keras import layers  
from tensorflow.keras import models  
from tensorflow.keras.datasets import imdb
```

---

### Load Dataset

```
vocab_size = 10000

max_length = 200

(X_train, y_train), (
    X_test,
    y_test
) = imdb.load_data(
    num_words=vocab_size
)
```

---

## Pad Sequences

The reviews have different lengths, so we convert them to a fixed length.

```
X_train = tf.keras.utils.pad_sequences(
    X_train,
    maxlen=max_length,
    padding="post",
    truncating="post"
)

X_test = tf.keras.utils.pad_sequences(
    X_test,
    maxlen=max_length,
    padding="post",
    truncating="post"
)
```

---

## Build LSTM Model

```
model = models.Sequential([

    layers.Embedding(
        input_dim=vocab_size,
        output_dim=64,
        input_length=max_length
    ),

    layers.LSTM(64),
```

```
layers.Dense(  
    1,  
    activation="sigmoid"  
)  
)
```

---

## Compile Model

```
model.compile(  
    optimizer="adam",  
    loss="binary_crossentropy",  
    metrics=["accuracy"]  
)
```

---

## Display Model Architecture

```
model.summary()
```

---

## Train Model

```
history = model.fit(  
    X_train,  
    y_train,  
    epochs=3,  
    batch_size=64,  
    validation_split=0.2  
)
```

---

## Evaluate Model

```
test_loss, test_accuracy = (  
    model.evaluate(  
        X_test,  
        y_test  
    )
```

```
)  
  
print(  
    "Test Accuracy:",  
    test_accuracy  
)
```

## Model Architecture:

Text / Tokenized Review



Embedding Layer



LSTM Layer



Dense Layer



Sigmoid



Positive / Negative

## Explanation of Layers

### 1. Embedding Layer

Converts integer word IDs into dense vectors.

### 2. LSTM Layer

Processes the sequence and learns information from previous tokens.

### 3. Dense Layer

Produces the final binary classification output.

### 4. Sigmoid Activation

Produces a value between 0 and 1 that can be interpreted as the estimated probability of the positive class.

## Easy Definition:

**An LSTM sentiment model converts words into embeddings, processes their sequence using LSTM, and predicts positive or negative sentiment using a Dense output layer.**

---

## PART B: ATTENTION CALCULATION

### Given Attention Scores:

[1.2, 0.5, 2.0]

We need to calculate the attention weights using **Softmax**.

#### Softmax Formula:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Given:

$$x = [1.2, 0.5, 2.0]$$

#### Step 1: Calculate Exponentials

$$\begin{aligned} e^{1.2} &\approx 3.3201 & e^{0.5} &\approx 1.6487 \\ e^{2.0} &\approx 7.3891 \end{aligned}$$

#### Step 2: Calculate Sum

$$3.3201 + 1.6487 + 7.3891 \approx 12.3579$$

#### Step 3: Calculate Attention Weights

For the first position:

$$\frac{3.3201}{12.3579} \approx 0.2687$$

For the second position:

$$\frac{1.6487}{12.3579} \approx 0.1334$$

For the third position:

$$\frac{7.3891}{12.3579} \approx 0.5979$$

#### Final Attention Weights:

[0.2687, 0.1334, 0.5979]

**Sum:**

$0.2687 + 0.1334 + 0.5979 \approx 10.2687 + 0.1334 + 0.5979 \approx 1$

Therefore:

Position 1  $\rightarrow$  0.2687

Position 2  $\rightarrow$  0.1334

Position 3  $\rightarrow$  0.5979

**Highest Attention:**

The **third input position** receives the highest attention because:

$0.5979 > 0.2687 > 0.1334$

Therefore:

**Highest attention = Position 3**

**Python Verification:**

```
import numpy as np
```

```
scores = np.array([
    1.2,
    0.5,
    2.0
])
```

```
exp_scores = np.exp(
    scores
)
```

```
attention_weights = (
    exp_scores /
    np.sum(exp_scores)
)
```

```
print(
    "Attention Weights:",
    attention_weights
)
```

```
print(
```

```
"Highest Attention Position:",  
np.argmax(  
    attention_weights  
) + 1  
)
```

### Expected Output:

Attention Weights:  
[0.26866364 0.13341442 0.59792194]

Highest Attention Position:  
3

---

## COMPLETE REVISION TABLE

| Topic               | Key Point                                          |
|---------------------|----------------------------------------------------|
| NLP                 | Processing and understanding human language        |
| NLP Pipeline        | Cleaning → Tokenization → Features → Model         |
| Lowercasing         | Converts text to lowercase                         |
| Punctuation Removal | Removes punctuation marks                          |
| Stopwords           | Common words that may be removed depending on task |
| Tokenization        | Splits text into tokens                            |
| Stemming            | Produces a root-like form                          |
| Lemmatization       | Produces a meaningful base form                    |
| Unigram             | One-word sequence                                  |
| Bigram              | Two-word sequence                                  |
| Trigram             | Three-word sequence                                |
| NLTK                | NLP learning and linguistic processing             |

|                |                                                        |
|----------------|--------------------------------------------------------|
| spaCy          | Fast production-oriented NLP                           |
| Gensim         | Word embeddings and topic modeling                     |
| BoW            | Word-count representation                              |
| TF-IDF         | Weighted word representation                           |
| Word Embedding | Dense vector representation                            |
| RNN            | Basic recurrent neural network                         |
| LSTM           | RNN with gates for long-term dependencies              |
| GRU            | Gated recurrent model with simpler architecture        |
| Seq2Seq        | Converts one sequence into another                     |
| Encoder        | Processes input sequence                               |
| Decoder        | Generates output sequence                              |
| Context Vector | Representation passed from encoder to decoder          |
| Attention      | Focuses on relevant input positions                    |
| Accuracy       | Overall correct predictions                            |
| Precision      | Correct positive predictions among predicted positives |
| Recall         | Correct positives among actual positives               |
| F1             | Harmonic mean of precision and recall                  |
| Perplexity     | Common language-model evaluation measure               |

## IMPORTANT FORMULAS

### Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

### Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

## Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

## F1-Score

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

## TF-IDF

$$\text{TFIDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

## Softmax

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

# CONCLUSION

Natural Language Processing enables computers to process and analyze human language. Text preprocessing techniques such as tokenization, stemming, lemmatization, stopword removal, and normalization prepare raw text for Machine Learning and Deep Learning models.

Libraries such as **NLTK**, **spaCy**, and **Gensim** provide tools for different NLP tasks. **Bag of Words** and **TF-IDF** provide traditional text representations, while **word embeddings** provide dense vector representations.

For sequential NLP tasks, **RNN**, **LSTM**, and **GRU** can process information across a sequence. **Seq2Seq models** are useful for converting one sequence into another, while **attention mechanisms** allow models to focus on relevant parts of the input.

Finally, LSTM models can be used for applications such as sentiment classification, while attention weights can identify which input positions receive greater focus during sequence processing.